SIMATS ENGINEERING

DEPARTMENT OF INFORMATION TECHNOLOGY

ASSIGNMENT 3 – CO3

SYNTAX-DIRECTED TRANSLATION

Synthesized and Inherited Attributes, Dependency Graphs,
SDT Schemes and Attribute-Based Tree Construction

Particulars	**Details**
Student A	Balaji Rohith MV | Register No.: 192521269
Student B	Yogeshwaran | Register No.: 192524363
Department / Section	Information Technology
Academic Year	2025–2026
Duration / Total Marks	1 Week / 50 Marks per student
Assignment Mode	Pair Work – both students participate in every question

CO3: Analyze syntax-directed translation using synthesized and inherited attributes to generate annotated parse trees, dependency graphs, syntax-directed translation schemes and intermediate representations.

Submission note: The answers below provide worked derivations, semantic rules, trace examples, diagrams, test designs and implementation blueprints. Numerical outputs explicitly labelled as hand-derived or illustrative are not presented as measurements from an executed program. Run the supplied implementation logic and attach genuine terminal output before claiming experimental execution.

Q1 – Synthesized Attributes and the Annotated Parse Tree (6 Marks)

1.1 Syntax-Directed Definition (SDD)

A syntax-directed definition is a context-free grammar augmented with attributes and semantic rules. Attributes carry information through the parse tree. A synthesized attribute is computed at a node from attributes of its children. For arithmetic expressions, E.val stores the numeric value represented by the expression.

```
E → E1 + T { E.val = E1.val + T.val }
E → E1 - T { E.val = E1.val - T.val }
E → T { E.val = T.val }
T → T1 * F { T.val = T1.val * F.val }
T → T1 / F { T.val = T1.val / F.val }
T → F { T.val = F.val }
F → ( E ) { F.val = E.val }
F → digit { F.val = digit.lexval }
```

The grammar encodes precedence structurally: E handles addition and subtraction; T handles multiplication and division; F handles parentheses and digits. Left recursion makes +, −, *, and / left-associative. Division by zero must be detected as a semantic/runtime error rather than allowed to produce an invalid numeric value.

1.2 Parse tree and annotated values

For 3 * (4 + 5) - 6 / 2, evaluate the parenthesized addition first, then multiplication and division, and finally subtraction. The arithmetic is 3 × 9 − 3 = 24.

Subexpression / node	Semantic rule	Computed value
4 + 5	E.val = 4 + 5	9
(4 + 5)	F.val = E.val	9
3 * (4 + 5)	T.val = 3 × 9	27

6 / 2 T.val = 6 / 2 3
3 * (4 + 5) - 6 / 2 E.val = 27 − 3 24

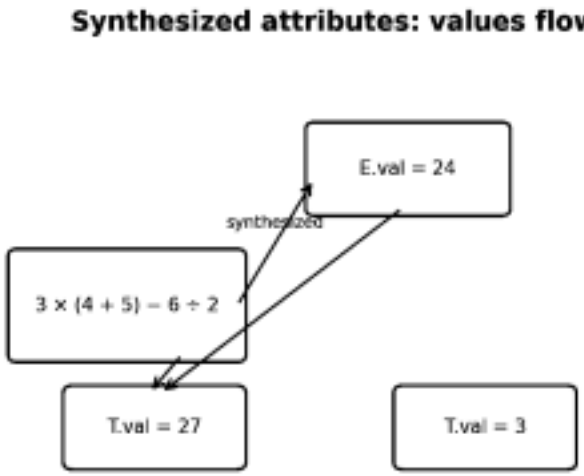


Figure 1. Synthesized values are calculated in child-to-parent direction; the root contains the final expression value.

1.3 Why the SDD is S-attributed

Every attribute in this definition is synthesized: E.val depends only on child E/T values, T.val depends only on child T/F values, and F.val depends only on the child E or the digit token's lexical value. No production requires information from a parent or sibling. Therefore the definition is S-attributed.

The natural evaluation order is post-order: evaluate children before their parent. This order is forced by the dependency edges. A parent value cannot be computed until the child values it uses are available. A depth-first post-order traversal therefore evaluates every node exactly once.

1.4 Student A – annotated tree design

The annotated tree should label every nonterminal with its value. For the expression, the important internal annotations are: the inner E for (4+5) has value 9; the multiplication T has value 27; the division T has value 3; and the root E has value 24. Digit leaves carry lexical values 3, 4, 5, 6 and 2.

1.5 Student B – test plan and complexity evidence

Test input	Expected result / behaviour	Purpose
7	7	Single digit

2+3	5	Addition
9-4-1	4	Left associativity
2*3+4	10	Precedence
8/2/2	2	Left-associative division
(2+3)*4	20	Nested parentheses
3*(4+5)-6/2	24	Mixed operators
((1+2)*3)	9	Nested brackets
5/(3-3)	Division-by-zero error	Semantic/runtime error
2+*3	Syntax error	Malformed expression

For a binary expression tree with n operator/operand tokens, the number of parse-tree nodes is Θ(n), and one synthesized attribute evaluation per nonterminal gives Θ(n) attribute computations. A parser that constructs one node per grammar symbol also uses Θ(n) space. The exact node count depends on whether punctuation and grammar-only nonterminals are counted.

Expression	**Approx. token count**	**Expected growth pattern**
7	1	Constant small tree
2+3	3	Linear increase
2+3*4	5	Linear increase
(2+3)*(4+5)	9	Linear increase
3*(4+5)-6/2	11	Linear increase

For a reproducible experiment, instrument the parser with counters: increment nodeCount whenever a parse-tree node is created and attributeCount whenever a semantic rule computes a value. Run the test set, record actual counts, and compare the ratios as input size increases. Do not substitute estimated counts for measured output.

1.6 Part B – height as a second synthesized attribute

```
E → E1 + T { E.val = E1.val + T.val; E.height = 1 + max(E1.height,T.height) } E
→ E1 - T { E.val = E1.val - T.val; E.height = 1 + max(E1.height,T.height) } E →
T { E.val = T.val; E.height = T.height }
T → T1 * F { T.val = T1.val * F.val; T.height = 1 + max(T1.height,F.height) } T
→ T1 / F { T.val = T1.val / F.val; T.height = 1 + max(T1.height,F.height) } T →
F { T.val = F.val; T.height = F.height }
F → ( E ) { F.val = E.val; F.height = E.height }
F → digit { F.val = digit.lexval; F.height = 1 }
```

Height is the number of nodes on the longest root-to-leaf path under the convention that a leaf has height 1. In a left-recursive grammar, a long chain such as a-b-c-d forms a left-deep parse tree and can have height proportional to the number of operators. A right-recursive grammar produces a right-deep chain instead; its height is also linear for an unbalanced chain. The side of the skew changes, not the asymptotic height.

A balanced expression tree may have logarithmic height, but ordinary left- or right-recursive grammar derivations do not automatically balance the tree. The height experiment should therefore plot measured height against token count for left chains, right chains and balanced parenthesized expressions.

Q2 – Inherited Attributes in Declarations (7 Marks)

2.1 SDD for declaration type propagation

The declaration grammar is D → T L; T → int | float; L → L , id | id. T.type is synthesized from the type keyword. L.in is inherited: the parent declaration passes the selected type into the identifier-list subtree.

```
D → T L { L.in = T.type }
T → int { T.type = integer }
T → float { T.type = real }
L → L1 , id { L1.in = L.in; addType(id.entry, L.in) }
L → id { addType(id.entry, L.in) }
```

The semantic action addType(entry,type) inserts an identifier into the symbol table with its declared type. It must check whether the identifier already exists in the

current scope before insertion. If it exists, report a duplicate declaration and do not silently overwrite the old entry.

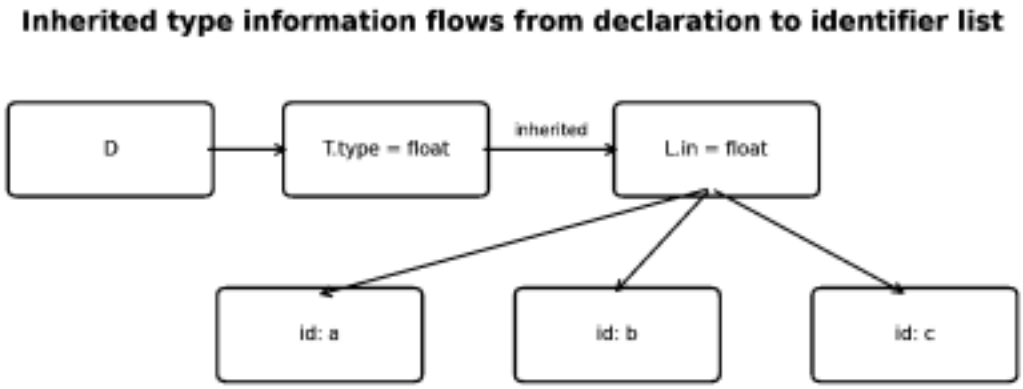


Figure 2. T.type is synthesized at T and passed downward as L.in; the list distributes the type to each identifier.

2.2 Annotated example: float a, b, c

Node	Attribute / action	Result
T → float	T.type	real (floating-point)
D → T L	L.in = T.type	L.in = real
L → L1 , c L1 → L2 , b	L1.in = L.in; addType(c,real) L2.in = L1.in; addType(b,real)	c : real b : real

L2 → a addType(a,real) a : real

Final symbol table: a → real, b → real, c → real. The inherited attribute follows the declaration context and reaches every identifier leaf.

2.3 Student A – why L.in is inherited

L.in cannot be synthesized in this grammar because the list itself does not determine the type. The type is selected by the parent declaration through T. Information therefore flows from parent to child. In the left-recursive production L → L1 , id, the same inherited type is also passed from the current list node to its left child, while the current production assigns the type to the rightmost id.

Required evaluation order: first compute T.type from the keyword; then assign L.in; then propagate L.in through the list and execute addType for each identifier. This is a depth-first, left-to-right-compatible inherited-attribute evaluation.

2.4 Left- and right-recursive alternatives

```
Left recursive: L → L1 , id { L1.in=L.in; addType(id,L.in) }
 L → id { addType(id,L.in) }
Right recursive: L → id , L1 { addType(id,L.in); L1.in=L.in }
 L → id { addType(id,L.in) }
```

Both versions pass the same type context to every identifier. The left-recursive form processes the final identifier at the current node and propagates to the left child. The right-recursive form processes the first identifier and propagates to the right child. A top-down parser often prefers the right-recursive or repetition form to avoid left recursion.

2.5 Student B – synthesized versus inherited

Feature	**Synthesized attribute**	**Inherited attribute**
Information flow	Child → parent	Parent or left sibling → child
Typical use	Expression value, result type	Declaration type, expected type, scope context
Evaluation	After children are known	Before or during child traversal
Example	E.val = E1.val + T.val	L.in = T.type
Common strategy	Post-order	Depth-first left-to-right for L-attributed SDDs

The same information can sometimes be transmitted either way by changing the grammar. A declaration type can be synthesized by a redesigned list node if the type token is structurally incorporated into each variable declaration. This can simplify propagation but may duplicate type information or change the grammar.

If a semantic rule reads an attribute before it has been computed, the evaluator has an unresolved dependency. It must not use a default or stale value. A correct

evaluator delays the action until dependencies are ready or reports an ill-defined attribute dependency.

Duplicate declarations are detected during addType. Undeclared-variable errors are detected later when an identifier is used and symbol-table lookup fails. Declaration-time inherited attributes provide the type context; the symbol table records the result.

2.6 Joint design: per-variable types

For declarations such as int a, float b, the type belongs to each declarator rather than to a single shared list. Use a grammar such as Decl → Type id | Type id ; Decl, or a list of typed declarators. Each declarator can synthesize or locally receive its own type. The shared inherited attribute L.in for a homogeneous identifier list is no longer needed because each variable carries its own type specifier.

Input	**Expected symbol-table effect**
float a, b, c;	Insert a:real, b:real, c:real
int count;	Insert count:integer
int a; float a; float x; x = 2.5;	Duplicate declaration error for a Valid declared use

float x; y = 1; Undeclared identifier y

Q3 – Dependency Graphs and Evaluation Order (7 Marks)

3.1 Meaning of a dependency graph

An attribute dependency graph has one vertex for each attribute instance in an annotated parse tree. A directed edge u → v means that v cannot be evaluated until u is known. A valid evaluation order is any topological ordering of this directed acyclic graph (DAG).

For float a, b, the type keyword determines T.type, which determines L.in, which

determines the inherited type at the list nodes and the type inserted for each identifier. For 2 + 3 * 4, literal values flow upward through multiplication and then addition.

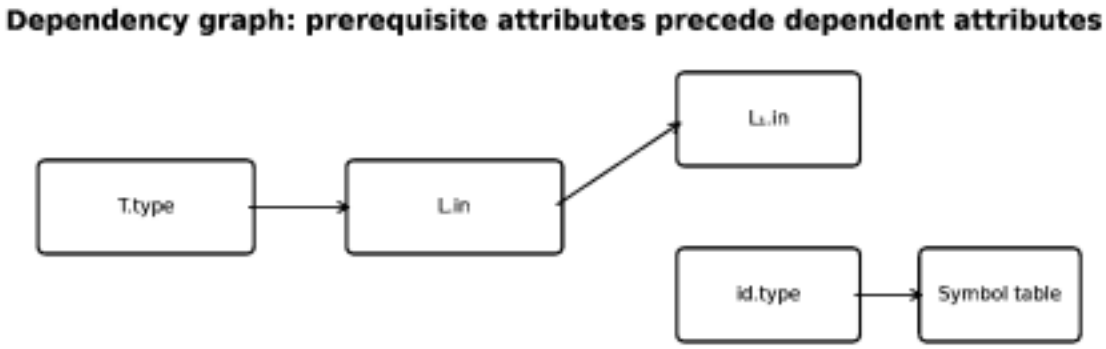


Figure 3. A dependency graph represents prerequisite relationships; any topological ordering is valid. **3.2**

Example evaluation orders

Example	Dependencies	One valid order
float a,b	T.type → L.in → identifier type insertions	T.type, L.in, a.type, b.type
2 + 3 * 4	3.val and 4.val → product.val; 2.val and product.val → root.val	3.val, 4.val, 2.val, product.val, root.val

Different topological orders may exist when two attributes are independent. For example, the literal values 2 and 3 can be evaluated in either order. The final result remains 14 because both orders satisfy all dependency edges and the semantic functions are deterministic.

3.3 Generic attribute evaluator

```
build_dependency_graph(parse_tree):
 create one vertex for every attribute instance
 for each semantic rule v := f(u1, ..., uk):
 add edge ui -> v for every prerequisite ui
 compute indegree of every vertex
 queue = all vertices with indegree 0
 order = []
```

```
while queue is not empty:
u = remove one vertex from queue
evaluate its semantic rule; append u to order
for each edge u -> v:
indegree[v] -= 1
if indegree[v] == 0: add v to queue
if len(order) != number_of_vertices: report cyclic SDD
otherwise return evaluated attributes
```

Kahn's topological-sort algorithm runs in O(V+E), where V is the number of attribute instances and E is the number of dependency edges. The attribute functions add their own execution cost.

3.4 Student A – cycle example and detection

Consider the deliberately invalid rules A.x = B.y and B.y = A.x. The dependency graph contains A.x → B.y and B.y → A.x, forming a cycle. Neither attribute has an initial prerequisite-free value, so no topological order exists. Kahn's algorithm stops with fewer than V vertices processed; the remaining vertices identify the cyclic dependency region.

Cycle test	Indegree-zero queue	Processed count	Conclusion
A.x ↔ B.y	Initially empty	0 of 2	Cycle detected; SDD is not well-defined

A valid numbering of a dependency graph is any topological numbering in which every arrow points from a lower number to a higher number. If two independent leaves are both ready, swapping their order gives another valid numbering.

3.5 Student B – S-attributed and L-attributed definitions

Class	Formal condition	Evaluation strategy
S-attributed	All attributes are synthesized	Post-order traversal
L-attributed	For A → X1…Xn, each inherited attribute of Xi may depend only on inherited attributes of A and attributes of X1…Xi−1; synthesized attributes of A depend on child attributes	Depth-first, left-to-right

Q1 is S-attributed and therefore also L-attributed: its attributes are synthesized only. Q2 is L-attributed because L.in is passed from the parent and propagated in a left-to-right-compatible way; it is not S-attributed because it contains an inherited attribute.

An L-attributed but not S-attributed example is the declaration SDD in Q2. A non-L-attributed example is A → B C with B.in = C.syn when C is to the right of B: the inherited attribute of B depends on a right sibling, violating the left-to-right condition.

SDD category	**Suitable evaluation**
S-attributed	Post-order
L-attributed	Depth-first, left-to-right

General acyclic attribute grammar Topological dependency evaluation

Restricting an SDD to L-attributed form gives a predictable one-pass, left-to-right evaluation strategy and supports efficient recursive-descent parsing. General attribute grammars are more expressive, but may require global dependency construction, scheduling and cycle detection.

Q4 – Syntax-Directed Translation Schemes (7 Marks)

4.1 SDD versus SDT

An SDD declaratively specifies attributes and semantic equations. An SDT embeds executable actions directly into grammar productions. The SDD describes what values mean; the SDT specifies when the actions execute during parsing.

4.2 Infix-to-postfix SDD and SDT

For infix-to-postfix translation, operands are emitted when recognized and operators are emitted after their operands. This is achieved by placing the operator action at the end of the corresponding production.

```
E → E + T { print('+') }
E → E - T { print('-') }
E → T
T → T * F { print('*') }
T → T / F { print('/') }
T → F
F → ( E )
F → id { print(id.lexeme) }
```

For a + b * (c - d), the parser respects precedence and parentheses. The output is a b c d - * +. The multiplication action runs after the right factor has been translated; the addition action runs after the complete right term.

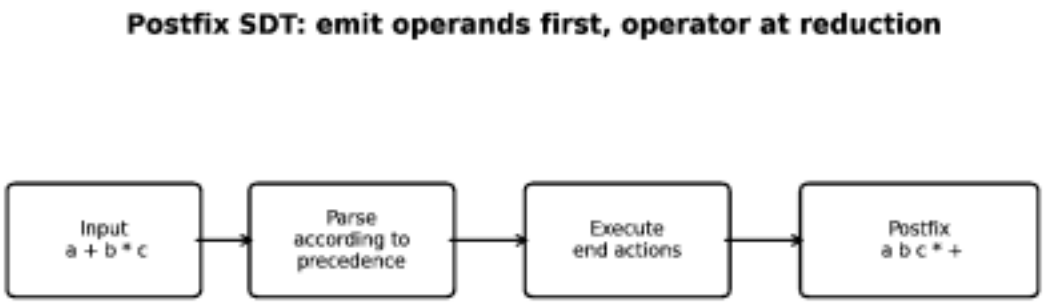


Figure 4. The SDT emits each operator only after the operand translations are complete.

4.3 Student B – action placement and execution

Production	Action position	Reason
E → E + T	After T	Both operands have been emitted before +
T → T * F	After F	Both operands have been emitted before *
F → id	After id	Emit operand as soon as it is recognized
F → (E)	No output action	Parentheses control grouping but

		are not postfix operands

During a bottom-up parse, a reduction occurs after the right-hand side has been recognized. An action at the end of the production can therefore execute at reduction time. In a bottom-up implementation, semantic values or output fragments are stored alongside grammar symbols on the parser stack.

If the action for E → E + T is moved before T is translated, the output may become a + b c * rather than a b c * +. This is not postfix order: the operator appears before its right operand.

4.4 Student A – tests and action counts

Infix input	**Expected postfix**	**What it verifies**
a+b	a b +	Addition
a+b*c	a b c * +	Precedence
(a+b)*c	a b + c *	Parentheses
a-b-c	a b - c -	Left associativity
a/b/c	a b / c /	Left associativity
a*(b+c)-d	a b c + * d -	Combined grouping

For a valid expression with k binary operators and m operands, a postfix translation emits m operand actions and k operator actions, for m+k output actions. Since a binary expression has k=m−1, the total number of output actions is 2m−1, which is linear in expression size.

Q1 evaluates numeric attributes after or during tree construction; Q4 emits a translation string as actions execute. Both rely on grammar structure for precedence and associativity, but the output attribute differs: Q1 returns a value, while Q4 produces postfix tokens.

4.5 Removing left recursion

A predictive parser may use E → T E′ and E′ → + T {emit('+')} E′ | − T {emit('−')} E′ | ε. T is similarly rewritten with T′. The action must be placed after the operand T and before processing the remaining E′. For left-associative operators, a translation accumulator or inherited output fragment is often needed to preserve the same left-to-right grouping.

4.6 Joint engineering decision

On-the-fly SDT is preferable when the translation is local, actions can safely run at reduction points, and a complete parse tree is not needed later. Building a full tree first is preferable when later phases need multiple traversals, optimization, source-level diagnostics, or transformations that depend on distant context. Error recovery must also be considered: irreversible output emitted before a later syntax error may need buffering or rollback.

Q5 – Generating the Parse Tree / Syntax Tree Using Attributes (6 Marks)

5.1 AST construction SDD

An abstract syntax tree (AST) stores the essential operators and operands while omitting grammar-only nodes such as E, T, F and punctuation. The synthesized attribute node holds a pointer/reference to the AST node representing the subtree.

```
E → E1 + T { E.node = mkNode('+', E1.node, T.node) }
E → E1 - T { E.node = mkNode('-', E1.node, T.node) }
E → T { E.node = T.node }
T → T1 * F { T.node = mkNode('*', T1.node, F.node) }
T → T1 / F { T.node = mkNode('/', T1.node, F.node) }
T → F { T.node = F.node }
F → ( E ) { F.node = E.node }
F → id { F.node = mkLeaf(id.lexeme, id.entry) }
```

For assignment, add S → id = E { S.node = mkNode('=', mkLeaf(id), E.node) }.

The node constructor creates an operator node with pointers to its children; the leaf constructor stores the identifier name and symbol-table entry.

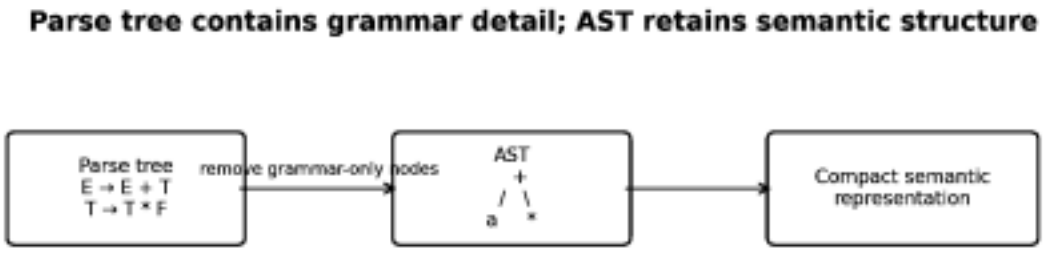


Figure 5. The AST removes grammar scaf olding while preserving operators and operand relationships.

5.2 Student A – pointer flow and recursion

Each child synthesizes a node pointer. The parent semantic action receives those pointers and creates a new parent node. The pointer is then synthesized upward until the root attribute points to the complete AST. This avoids copying entire subtrees; only references are passed.

For a left-recursive rule E → E1 + T, the AST node is mkNode('+',E1.node,T.node). After eliminating left recursion, use E → T E′ and E′ → + T E′ | ε. E′.inh carries the partial AST built so far; on + T, update E′.inh = mkNode('+', E′.inh, T.node), and on ε return E′.syn = E′.inh. The inherited accumulator preserves the left-associated structure.

5.3 Step-by-step AST for a = b + c * d

Step	Operation	AST fragment
1	Create leaves	a, b, c, d
2	Build multiplication	*(c,d)
3	Build addition	+(b, *(c,d))
4	Build assignment	=(a, +(b, *(c,d)))

```
=
/ \
a +
/ \
b *
/ \
c d
```

The multiplication node is constructed before the addition node because * has higher precedence. The assignment node is the root.

5.4 Student B – parse-tree versus AST evidence

Representation	Contains	Typical node growth
Concrete parse tree	All grammar nonterminals, terminals and punctuation	Θ(n)
AST	Operators, operands and required semantic constructs	Θ(n), with fewer nodes and lower constant factor

AST construction does not generally change asymptotic growth for ordinary expression grammars: both structures grow linearly with input length. The AST is more compact because it removes grammar-only nodes, parentheses tokens and intermediate nonterminals. The exact savings depend on the grammar and whether the parse tree counts punctuation and helper nodes.

The top-down, left-recursion-eliminated grammar uses E′.inh because the accumulated left-hand expression must be passed into the recursive tail. The bottom-up left-recursive grammar can synthesize E.node directly from completed children. Experimental evidence should count actual nodes for identical input strings in both representations; a smaller AST node count does not imply the same percentage saving for every input.

Q6 – Advanced Attribute Evaluation Mini-Investigations (7 Marks)

6.1 Student A – inherited attributes in bottom-up parsing

An LR parser recognizes the right-hand side of a production from the stack and reduces it to the left-hand side. Synthesized attributes are convenient because the value of the left-hand side is computed from values already on the stack for the right-hand side. Inherited attributes are harder: their value may come from a parent or left context that is not naturally represented as a child value at the reduction point.

A marker nonterminal is an empty production inserted at the point where the needed context is available. Its semantic action executes during the reduction of ε and stores the inherited value in a stack attribute or parser-side context.

```
D → T M L
M → ε { M.in = T.type }
T → int { T.type = integer }
T → float { T.type = real }
L → L , id { addType(id.entry, M.in) }
L → id { addType(id.entry, M.in) }
```

The marker does not add or remove any terminal strings: M derives ε, so D → T M L yields the same terminal sequence as D → T L. It changes the parser's internal action timing, not the language. A risk is incorrect stack-offset assumptions or an action firing at an unexpected point after a grammar change.

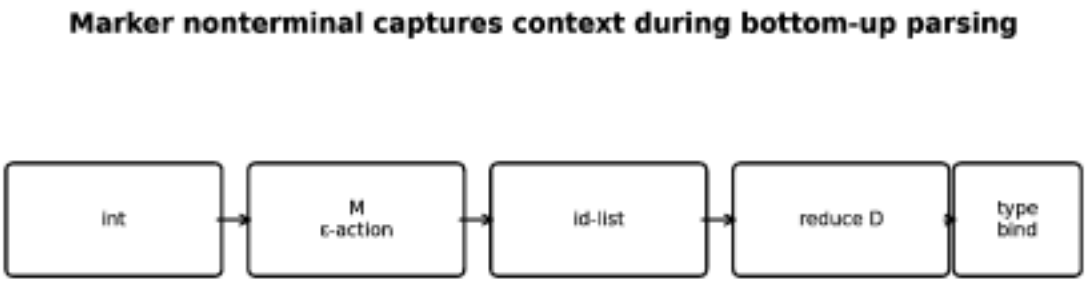


Figure 6. The marker action captures T.type before the identifier list is reduced.

6.2 Illustrative stack trace for int x, y

Parser event	Stack (symbols with attributes)	Semantic effect
Shift int	int	Recognize type keyword
Reduce T → int	T(type=integer)	Synthesize T.type
Reduce M → ε	T(type=integer) M(in=integer)	Capture inherited type
Shift x , y	T M id(x) , id(y)	Identifier tokens collected
Reduce L → id	T M L(x)	addType(x,integer)
Reduce L → L , id	T M L(x,y)	addType(y,integer)
Reduce D → T M L	D	Declaration complete

In an implementation, the action should access the semantic value associated with T through a stable parser-stack reference or an explicitly stored context object. A hard-coded offset is safe only if the parser state and grammar guarantee that the type value is always at that position. If optional productions or nested constructs alter the stack shape, the offset can become wrong; use named semantic values or a context structure instead.

6.3 Student B – three-address code SDD

Three-address code (TAC) represents each operation using at most three addresses. E.addr identifies where the expression result is stored; E.code contains the ordered TAC statements that compute it.

```
E → E1 + E2 { t=newTemp(); E.addr=t;
E.code=E1.code||E2.code||emit(t,'+',E1.addr,E2.addr) }
E → E1 * E2 { t=newTemp(); E.addr=t;
E.code=E1.code||E2.code||emit(t,'*',E1.addr,E2.addr) }
E → id { E.addr=id.name; E.code=ε }
```

For x = a + b * c - d, multiplication must be computed before addition, then subtraction, then assignment.

Step	Three-address instruction	Meaning
1	t1 = b * c	Compute product
2	t2 = a + t1	Add a to product

3	t3 = t2 - d	Subtract d
4	x = t3	Assign final value

newTemp() returns a fresh temporary name on every call (t1, t2, t3, …). The numbering is monotonic within a procedure or compilation unit, ensuring that two simultaneously live results are not accidentally assigned the same temporary.

6.4 Inherited labels for if-else

```
S → if ( B ) S1 else S2
 { B.true = newLabel(); B.false = newLabel(); S1.next = newLabel();
S2.next = S.next;
 S.code = B.code || label(B.true) || S1.code ||
goto(S.next) || label(B.false) || S2.code || label(S.next) }
```

B.true, B.false and S.next are inherited because the enclosing control-flow context determines where each branch should jump. The condition does not independently know the label chosen by its parent statement. Synthesized attributes are still appropriate for completed code fragments and expression addresses.

6.5 Cross-verification

A marker nonterminal preserves the accepted language because it derives ε; its purpose is to schedule a semantic action at a useful parser-stack position. It introduces implementation risks: action timing, stack-layout dependence, and grammar-maintenance complexity.

Code-fragment concatenation order is as important as attribute values. If E1.code, E2.code and the operator instruction are concatenated in the wrong order, the generated code may use a temporary before it is assigned. Correct TAC must preserve the dependency and evaluation order represented by the expression tree.

Q7 – Design a Syntax-Directed Translator from First Principles (10 Marks)

7.1 Stage 1 – independent design proposals

Student A proposal: use a compact statement grammar with declarations, assignments, if-else and while. Use inherited attributes for the current scope and expected type, synthesized attributes for expression type, AST node and code fragment, and labels inherited from enclosing control-flow statements. Evaluate with a depth-first, left-to-right SDT.

Student B proposal: use a recursive-descent parser with a symbol table and explicit expression precedence levels. Build an AST first, then perform type checking and TAC generation in a second traversal. This separates parsing from code generation and simplifies diagnostics, but requires an additional tree traversal.

7.2 Stage 2 – design comparison record

Dimension	Student A proposal	Student B proposal	Final decision
Translation timing	Actions during parsing	Build AST then translate	Hybrid: parse/build AST, run local semantic actions; emit TAC after validation
Attributes	Inherited context + synthesized results	Mostly synthesized AST/type data	Use inherited scope/type/labels and synthesized node/type/code
Diagnostics	Immediate semantic checks	Centralized semantic pass	Collect errors with source locations; do not emit invalid TAC
Control flow	Inherited labels	Second-pass label generation	Inherited true/false/next labels in translation pass
Strength	Avoids some separate passes	Clear separation and easier recovery	Retain modularity and explicit attribute flow

7.3 Stage 3 – final MiniLang grammar

```
Program → DeclList StmtList
DeclList → DeclList Decl | ε
Decl → Type id ; | Type id , IdList ;
Type → int | float
```

```
IdList → id | IdList , id
StmtList → StmtList Stmt | ε
Stmt → id = Expr ;
 | if ( Bool ) Stmt else Stmt
 | while ( Bool ) Stmt
 | { StmtList }
Expr → Expr + Term | Expr - Term | Term
Term → Term * Factor | Term / Factor | Factor
Factor → ( Expr ) | id | int_literal | float_literal
Bool → Expr relop Expr | ! Bool | Bool && Bool | Bool || Bool
```

For an LL parser, eliminate left recursion in expression and Boolean productions and implement precedence levels as separate nonterminals. For LR parsing, the left-recursive expression form is suitable. The dangling-else ambiguity is avoided here by requiring an explicit else in the if production; if optional else is required, use matched/unmatched statement grammar or a documented nearest-if rule.

7.4 Attribute table

Attribute	Kind	Purpose
T.type	Synthesized	Type represented by int or float
Decl.scope	Inherited	Current lexical scope / symbol table
Expr.type	Synthesized	Type of an expression
Expr.node	Synthesized	AST pointer for expression
Expr.addr	Synthesized	Address/temporary holding expression result
Stmt.next	Inherited	Label reached after a statement
Bool.true / Bool.false	Inherited	Control-flow destinations for Boolean result
Stmt.code	Synthesized	Generated TAC for a statement
Decl.ok	Synthesized	Whether declaration was accepted

7.5 Semantic rules and type checking

```
Type → int { Type.type = INTEGER }
Type → float { Type.type = FLOAT }
Decl → Type id ; { insert(id, Type.type); Decl.ok = insertion_success }
Expr → Expr1 + Term
 { require_numeric(Expr1.type, Term.type);
```

```
 Expr.type = promote(Expr1.type, Term.type);
 Expr.node = mkNode('+', Expr1.node, Term.node);
 t = newTemp(); Expr.addr=t;
Expr.code=Expr1.code||Term.code||t=Expr1.addr+Term.addr }
Stmt → id = Expr ;
 { entry = lookup(id); if absent report undeclared;
 else check_assignable(entry.type, Expr.type);
 Stmt.code = Expr.code || id = Expr.addr }
```

The promotion function returns float if either operand is float; otherwise it returns int. Assignment is valid when the right-hand side can be assigned to the left-hand side under the language's conversion policy. This design permits int-to-float widening but rejects float-to-int narrowing unless an explicit cast is added.

7.6 Symbol table and error handling

Error	Detection point	Example diagnostic
Duplicate declaration	insert() finds existing name in current scope	Line 1: 'count' already declared
Undeclared variable	lookup() returns no entry at use	Line 4: 'total' is not declared
Type mismatch	assignment/operator type check	Cannot assign float expression to int variable
Invalid operator operands	operator semantic rule	Operator '*' requires numeric operands
Division by zero	constant-folding/runtime check	Division by zero in constant expression

A symbol-table entry should contain identifier name, declared type, scope depth, storage class if relevant, and source location. Entering a block creates a nested scope; leaving it removes or hides entries declared in that scope. Error recovery should continue after a declaration or statement boundary where possible.

7.7 Pseudocode for the translator

```
compile(source):
 tokens = lexer(source)
 ast = parser(tokens) # syntax errors stop invalid productions  symbolTable = new
ScopedSymbolTable()
 semanticCheck(ast, symbolTable) # declarations, uses, operand types  if
errors exist: print diagnostics; do not emit final TAC
```

```
else:
tac = translate(ast, newLabel, newTemp)
print annotated AST, symbol table, and TAC
return tac
```

7.8 Stage 4 – validation program

Use a complete program containing at least six statements so that declarations, arithmetic, conditionals and loops are all exercised:

```
int count;
float rate, total;
count = 3;
rate = 2.5;
total = rate * count + 1;
if (total > 5) count = count + 1; else count = 0;
while (count < 10) { count = count + 1; }
```

Formal validation: the expression attributes are synthesized from child expressions; scope and branch labels are inherited from enclosing constructs. For the chosen grammar, define the inherited attributes of each child using only the parent's inherited context and attributes of siblings to its left. The dependency graph is therefore acyclic and the SDD is L-attributed, provided code-generation labels are allocated before descending into the relevant child statements.

Complexity: lexical analysis and parsing are O(n) for a deterministic parser with bounded lookahead; each AST node is visited a constant number of times for semantic checking and TAC generation, also O(n). Symbol-table lookup is expected O(1) with hashing, or O(log s) with a balanced tree for s symbols. Overall expected time is O(n) with hashing, excluding unusually expensive diagnostics or optimization.

7.9 Student B – experimental validation matrix

Test	Input feature	Expected result
V1	Valid declarations and assignment	Accepted; symbol table and TAC produced
V2	Duplicate identifier	Duplicate-declaration diagnostic
V3	Use before declaration	Undeclared-variable diagnostic

V4 V5	int x; x = 2.5; Nested if-else inside while	Type error under no-narrowing policy Accepted; distinct labels and correct branches
V6	Parenthesized mixed arithmetic	Accepted; precedence respected
V7	Missing semicolon	Syntax error with source location
V8	Division by zero constant	Diagnostic or explicit runtime-check TAC

Compare the integrated translator with a baseline two-pass implementation: both must accept/reject the same programs and produce equivalent AST/TAC. Record token count, AST nodes, symbol lookups, semantic-rule evaluations, TAC instruction count and elapsed time over increasing program sizes. The assignment file does not provide executable source code or genuine timing logs for this new implementation, so timing values must be collected by running the implementation rather than invented.

7.10 Dependency graph and attribute evaluation

For a declaration, Type.type → Decl insertion and for an expression, child Expr.type values → parent Expr.type. For control flow, the parent statement allocates labels and passes Bool.true/Bool.false to the condition; the Boolean node synthesizes code that branches to those labels. A full dependency graph for a program is obtained by expanding these local dependencies for each AST node. No edge should point from a later sibling's attribute to an earlier sibling's inherited attribute.

7.11 Stage 5 – cross-defence

Student A should use the validation matrix, parser acceptance results, symbol-table contents, AST node counts and TAC equivalence to explain the evidence supporting the design. Student B should explain that synthesized attributes arise from information computed by child constructs (expression type, AST node, code

fragment), while inherited attributes arise from enclosing context (scope, expected type, branch labels and next label).

7.12 Stage 6 – joint evaluation

Syntax-directed translation is suitable because many compiler actions are closely tied to grammar productions: expression typing, AST construction, declaration insertion and control-flow code generation. Attributes avoid repeatedly rediscovering local relationships in separate passes. However, they introduce attribute storage, dependency management and parser-stack complexity. A separate pass is preferable when an action needs information from distant parts of the program, whole-program analysis, optimization requiring global facts, or multiple target-specific outputs.

Designer checklist: (1) identify the information needed by each construct; (2) classify each item as synthesized or inherited; (3) draw dependencies; (4) check acyclicity; (5) test whether S-attributed or L-attributed evaluation is sufficient; (6) choose parse-time or later execution; (7) define scope and error recovery; (8) validate with positive, negative and boundary tests.

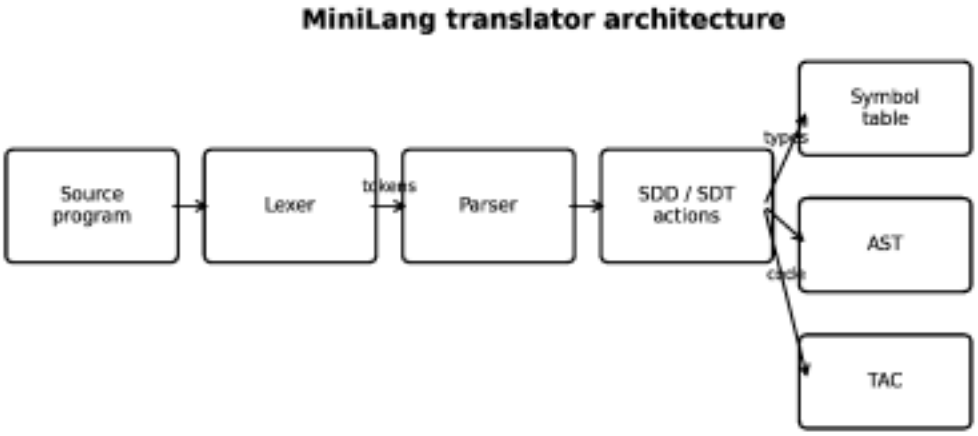


Figure 7. MiniLang translation pipeline: parsing and semantic actions coordinate the symbol table, AST and TAC output.

Contribution Record

Question	Student A – Primary contribution	Student B – Primary contribution	Joint work
Q1	Annotated tree, S-attributed proof, height experiment	Test expressions, instrumentation and output comparison	SDD and bottom-up evaluator
Q2	Inherited flow, recursion alternatives, symbol-table analysis	Synthesized/inherited comparison and error analysis	Declaration SDD and per-variable type design
Q3	Dependency graph, topological orders, cycle detection	S/L-attributed classification and evaluation strategies	Generic evaluator and dependency examples
Q4	Test design, action counts, engineering analysis	SDT action placement and correctness	Postfix translation implementation
Q5	AST SDD, pointer flow, inherited accumulator	Node-count comparison and memory analysis	AST constructors and parser integration
Q6	Marker nonterminal and LR stack analysis	TAC SDD and inherited control-flow labels	Cross-verification
Q7	Formal attribute/dependency validation	Test matrix, implementation instrumentation	Final grammar, translator design and evaluation

Primary contribution identifies responsibility for preparing and defending a component; it does not remove either student's responsibility to understand the complete assignment.

References

1. Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2006.

2. Cooper, K. D., and Torczon, L. Engineering a Compiler, 2nd ed., Morgan Kaufmann, 2011.

3. Appel, A. W. Modern Compiler Implementation in C, Cambridge University Press, 1998.

4. Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., and Langendoen, K. Modern Compiler Design, 2nd ed., Springer, 2012.